

The Dynamic Sporadic Server Algorithm for Aperiodic Task Scheduling in EDF Systems

Oleg Kelner
Hochschule Hamm-Lippstadt
Lippstadt, Germany
oleg.kelner@stud.hshl.de

Abstract—In real-time embedded systems, the joint scheduling of hard periodic tasks and soft aperiodic requests remains a critical challenge. While periodic tasks ensure system stability, aperiodic requests often require rapid response times without compromising hard deadlines. This paper explores the Dynamic Sporadic Server (DSS) algorithm, an extension of the fixed-priority sporadic server adapted for Earliest Deadline First (EDF) environments. We define the problem of aperiodic responsiveness, establish the mathematical foundations for DSS schedulability, and provide a state-machine-based C++ implementation model. Finally, the practical utility of DSS is demonstrated through its application in multimedia control and robotic systems.

I. INTRODUCTION

Real-time systems are increasingly complex, often requiring the simultaneous execution of periodic activities (e.g., control loops) and sporadic or aperiodic activities (e.g., operator inputs or sensory data updates). Periodic tasks typically possess hard deadlines, meaning a single failure to meet them can result in catastrophic consequences. Aperiodic tasks, conversely, often have soft deadlines where timing is desirable but not strictly critical.

The historical approach of scheduling aperiodic tasks as background activities—executing only when no periodic tasks are ready—frequently leads to unacceptably high average response times. While polling servers improved this by allocating specific slots for aperiodic service, they suffer from inefficiency; if no aperiodic request is present at the start of a polling period, the allocated capacity is wasted. To address this, "bandwidth-preserving" algorithms like the Sporadic Server (SS) were developed. The Dynamic Sporadic Server (DSS) is a specific adaptation for systems utilizing the Earliest Deadline First (EDF) scheduling policy.

II. PROBLEM STATEMENT

The central problem addressed by the DSS algorithm is the optimisation of average response times for soft aperiodic tasks while guaranteeing 100% schedulability for hard periodic tasks. In a dynamic priority environment like EDF, priorities are assigned based on absolute deadlines. A simple server might "double hit" the system by demanding its full capacity at the beginning of every period, regardless of when it actually executes, which can cause periodic tasks to miss their deadlines.

Therefore, the DSS must implement a replenishment scheme that allows it to preserve its capacity until a request

arrives, effectively "shifting" its high-priority execution window to the moment of actual demand without overloading the processor. The challenge lies in defining a replenishment rule that maintains temporal isolation between the server and other tasks.

III. MATHEMATICAL FOUNDATIONS

A. Necessary Mathematical Basics

A DSS is defined by two primary parameters: a capacity C_s (the maximum execution budget) and a period T_s . The server utilization is given by $U_s = C_s/T_s$.

B. Formal Description of the Topic

1) *Deadline Assignment*: In an EDF environment, the server is treated as a periodic task with a dynamic deadline. When the server transitions from an idle state to a ready state at time t , it is assigned an absolute deadline d such that:

$$d = t + T_s \quad (1)$$

This deadline also defines the next replenishment time.

2) *Schedulability Analysis*: The inclusion of a DSS does not compromise the optimality of the EDF scheduler. A set of periodic tasks with utilization U_p and a DSS is schedulable if and only if the total processor utilization does not exceed unity:

$$U_p + U_s \leq 1 \quad (2)$$

where $U_p = \sum_{i=1}^n \frac{C_i}{T_i}$. This theorem ensures that as long as the server does not exceed its allocated bandwidth over any interval of length T_s , the periodic tasks remain safe.

IV. THE DYNAMIC SPORADIC SERVER ALGORITHM

The operation of the DSS is governed by a state machine consisting of three states: IDLE, READY, and EXE.

A. State Transitions

- **IDLE to READY**: Occurs when an aperiodic request enters the system and $C_s > 0$, or when capacity becomes greater than zero due to a replenishment. At this instant t , the deadline and next replenishment time are set to $t + T_s$.
- **READY to EXE**: Occurs when the server has the highest priority (shortest deadline) and there are pending aperiodic requests.

- **EXE to READY:** Occurs if the server is preempted by a periodic task with a shorter deadline.
- **EXE to IDLE:** Occurs if the aperiodic request is completed or if C_s is exhausted. A replenishment for the consumed capacity is scheduled to occur exactly T_s units after the server last entered the READY state from IDLE.

B. Replenishment Rule

Unlike a periodic server, DSS capacity is not replenished at the start of a period. Instead, each consumed portion of capacity is replenished exactly T_s units of time after the server became active to serve that specific portion. This prevents the "double-hit" problem and ensures the server behaves like a periodic task with period T_s and execution time C_s over the long term.

V. RUNTIME AND OVERHEAD ANALYSIS

The computational overhead of the DSS algorithm primarily resides in the management of the replenishment queue and the deadline assignment logic.

A. Algorithm Complexity

The selection of the highest priority task in an EDF system with N tasks typically requires $O(\log N)$ with a priority queue or $O(N)$ with a simple array. In our implementation, the 'select_task' function performs a linear scan, resulting in $O(N)$ complexity per scheduling point.

The replenishment rule requires $O(1)$ time to schedule a new replenishment (appending to the circular buffer) and $O(1)$ time to process a replenishment when the timer expires. Compared to the Total Bandwidth Server (TBS), which only requires a single deadline update per request, the DSS carries a slightly higher memory overhead due to the replenishment queue, but this is justified by the superior temporal isolation it provides.

VI. IMPLEMENTATION MODEL

The Dynamic Sporadic Server was implemented using UPPAAL Timed Automata. To ensure compatibility with standard model checking and avoid the undecidability associated with arbitrary stopwatch stop-clocks, a discrete-time modeling approach was adopted.

A. Model Structure

The model consists of four primary automaton templates:

- **PeriodicTask:** Models hard periodic tasks that trigger at regular intervals and request execution with a specific absolute deadline.
- **DSS:** Implements the server logic, including dynamic deadline assignment ($d = t + T_s$) and consumption tracking.
- **Replenisher:** Manages a FIFO queue of replenishment events, restoring budget to the DSS exactly T_s units after consumption began.
- **Ticker:** A centralized clock that synchronizes the system state and invokes the EDF scheduler at each time step.

B. EDF Scheduler Logic

The scheduler is implemented as a C-style function within the UPPAAL declaration. At each time step, it evaluates all ready tasks and assigns the CPU to the task with the smallest absolute deadline. This ensures that the dynamic priority nature of EDF is correctly captured.

VII. APPLICATION REALIZATION

The DSS implementation was validated against a task set consisting of two hard periodic tasks and one DSS. Formal verification using UPPAAL's verifier confirmed that the system is deadlock-free and that all hard periodic tasks meet their deadlines even under heavy aperiodic load.

The model is particularly useful in hierarchical embedded systems where soft applications, such as multimedia stream control, coexist with hard hardware control loops [1]. In robot control, periodic tasks ensure stability via sensor acquisition and low-level control, while the DSS manages operator requests or path planning updates.

A. Performance Comparison

Simulation results indicate that while DSS performs better than background or polling service, it may be outperformed by the Total Bandwidth Server (TBS) or Improved Priority Exchange (IPE) in terms of average response time for very heavy aperiodic loads. However, DSS provides better temporal isolation and protection during overloads than TBS. It is a balanced choice for designers who need to trade off implementation complexity against responsiveness.

VIII. CONCLUSION

The Dynamic Sporadic Server is a robust algorithm for managing aperiodic tasks in dynamic priority systems. By utilizing a replenishment rule tied to actual consumption, it provides rapid responsiveness while maintaining the hard guarantees of the underlying EDF schedule. Its low implementation complexity makes it a practical candidate for real-world real-time operating systems.

REFERENCES

- [1] M. Spuri and G. Buttazzo, "Scheduling aperiodic tasks in dynamic priority systems," *Real-Time Systems*, vol. 10, pp. 179–210, 3 1996. [Online]. Available: <http://link.springer.com/10.1007/BF00360340>